Next, you add an item to the cart, check out and fill in the pay type details, place your order, and verify that you get a *Thank you* response.

Since this test is about the user's experience end-to-end, you don't need to look at the jobs that have been queued—instead we need to make sure they are executed. It's sufficient to assert the *results* of those jobs having been executed. To that end, the method `perform_enqueued_jobs` will perform any jobs that get enqueued inside the block of code given to it.Since our `ChangeOrderJob` enqueues a mail job, clearing the queue once isn't enough, so we clear it twice. After this, we verify that exactly two jobs were executed.



Joe asks:

How Are Background Jobs Run in Development or Production?

When running the application locally, the background jobs are executed and emails are sent by Rails. By default, Rails uses an in-memory queue to manage the jobs. This is fine for development, but it could be a problem in production. If your app were to crash before all background jobs were processed or before emails were sent, those jobs would be lost and unrecoverable.

In production, you'd need to use a different *back end*, as detailed in the Active Job Rails Guide.[75] Sidekiq is a popular open source back end that works great.[76] Setting it up is a bit tricky since you must have access to a Redis database to store the waiting jobs.[77] If you're using Postgres for your Active Records, Queue Classic is another option for a back end that doesn't require Redis—it uses your existing Postgres database.[78]

Next, check that an order was created in the way you expect by locating the created order and asserting that the values provided in the checkout form were properly saved.

Lastly, you need to check that the mail was sent. In the test environment, Rails doesn't actually deliver mail but instead saves it in an array available via `ActionMailer::Base.deliveries`. The objects in there respond to various methods that allow you to examine the email.